Name: Arya Bailur	**Semester: V**	**Branch: AI&DS**
Roll No: 08	Batch: T11	Date: 05-10-2026

Experiment No. 09

Aim: To provision cloud infrastructure using Terraform.

LO Mapped: LO4 – Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

- **IAM User / Account ID:** Experiment7 / 702561770822
- **Target AWS Region:** ap-southeast-2 (Asia Pacific – Sydney)
- **Deployment Status:** Provisioned, verified and deprovisioned successfully

1. Aim / Objective

To write, validate and run Infrastructure as Code (IaC) configurations in HashiCorp Terraform that automatically create, configure, verify and finally destroy an Amazon EC2 virtual machine and its related resources on Amazon Web Services (AWS).

2. Theoretical Background

What is Infrastructure as Code (IaC)?

Infrastructure as Code is a way of managing and provisioning cloud resources – networks, virtual machines, load balancers, storage and so on – through human-readable definition files instead of clicking through a web console. Because the infrastructure is described in code, it can be version-controlled and reproduced exactly, which keeps environments consistent and makes them easy to scale.

Key Terraform Concepts and Lifecycle Commands

HashiCorp Terraform is an open-source IaC tool that describes infrastructure declaratively using the HashiCorp Configuration Language (HCL).

- **Declarative configuration (.tf):** files in which the required cloud resources are defined. Every .tf file in a working directory is read together as one configuration.
- **terraform init:** prepares the working directory, downloads the required provider plugins (e.g. hashicorp/aws v6.67.0) and writes the dependency lock file (.terraform.lock.hcl).
- **terraform validate:** checks the HCL files for syntax errors and internal consistency.
- **terraform plan:** compares the configuration with what currently exists in the cloud and produces an execution plan without changing anything.
- **terraform apply:** carries out the plan through the AWS APIs to create or modify resources, and records the result in terraform.tfstate.
- **terraform output:** prints the output values defined in the configuration from the state file.

- **terraform destroy:** removes every resource Terraform manages, so that no unnecessary charges are incurred.

Related AWS Concepts and Security

1. **AWS Identity and Access Management (IAM):** provides the permissions needed (AmazonEC2FullAccess, AmazonVPCFullAccess, AmazonSSMReadOnlyAccess) and the access keys used for programmatic CLI access.
2. **AWS Systems Manager (SSM) Parameter Store:** returns current, region-specific values such as the latest Canonical Ubuntu 22.04 AMI ID (/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id).
3. **AWS Service Control Policies (SCPs):** organisation-level guardrails (policy p-81tb9lnn) that enforce compliance – mandatory resource and volume tags (Environment = "Dev", Project = "Experiment7"), the allowed instance type (t3.micro) and an attached SSH key pair (vmb-key).

3. Architecture Diagram and Workflow

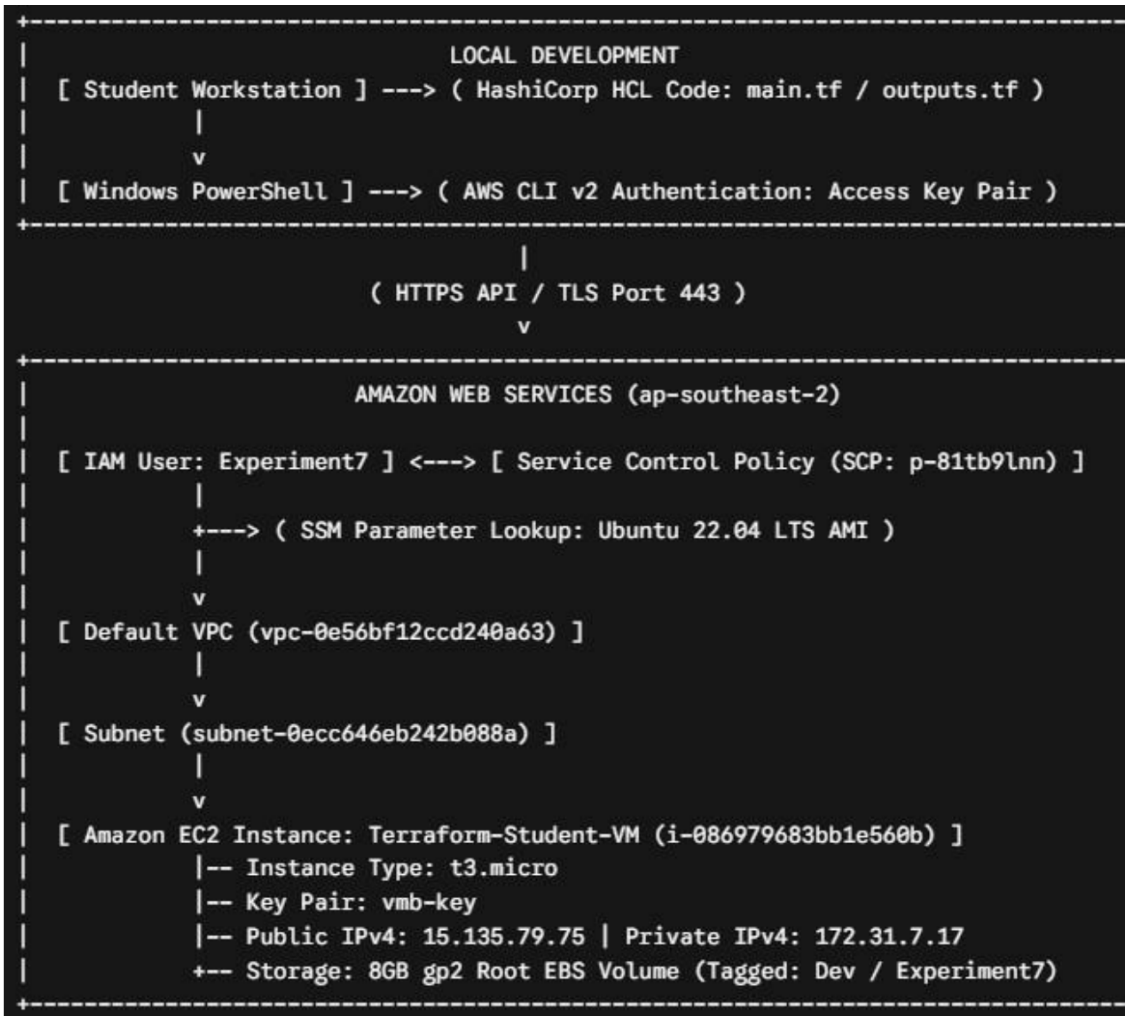


Fig. 1: Local Terraform workflow and the provisioned AWS architecture

4. Declarative Configuration Code

All configuration files were kept in the folder **C:\Terraform**.

main.tf (Infrastructure Definition)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-southeast-2"
}

# Look up the latest Ubuntu 22.04 LTS AMI from SSM Parameter Store
data "aws_ssm_parameter" "ubuntu_ami" {
  name = "/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id"
}

# Use the default VPC and its subnets
data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# EC2 instance
resource "aws_instance" "student_vm" {
  ami           = data.aws_ssm_parameter.ubuntu_ami.value
  instance_type = "t3.micro"
  subnet_id     = data.aws_subnets.default.ids[0]
  key_name      = "vmb-key"

  tags = {
    Name        = "Terraform-Student-VM"
    Environment = "Dev"
    Project     = "Experiment7"
  }

  volume_tags = {
    Name        = "Terraform-Student-VM"
    Environment = "Dev"
    Project     = "Experiment7"
  }
}
```

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-southeast-2"
}

data "aws_ssm_parameter" "ubuntu_ami" {
  name = "/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id"
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

resource "aws_instance" "student_vm" {
  ami           = data.aws_ssm_parameter.ubuntu_ami.value
  instance_type = "t3.micro"
  subnet_id     = data.aws_subnets.default.ids[0]
  key_name      = "vmb-key"

  tags = {
    Name        = "Terraform-Student-VM"
    Environment = "Dev"
    Project     = "Experiment7"
  }
```

Fig. 2: main.tf in the editor

```
  volume_tags = {
    Name        = "Terraform-Student-VM"
    Environment = "Dev"
    Project     = "Experiment7"
  }
}
```

Fig. 3: volume_tags block required by the SCP

outputs.tf (Output Values)

```
output "instance_id" {
  description = "ID of the created EC2 instance"
  value       = aws_instance.student_vm.id
}

output "instance_type" {
  description = "Instance type of the created VM"
  value       = aws_instance.student_vm.instance_type
}

output "public_ip" {
  description = "Public IP address assigned to the EC2 instance"
  value       = aws_instance.student_vm.public_ip
}
```

```
File    Edit    View

output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.student_vm.id
}

output "public_ip" {
  description = "Public IP address, if assigned"
  value       = aws_instance.student_vm.public_ip
}

output "instance_type" {
  value       = aws_instance.student_vm.instance_type
}
```

Fig. 4: outputs.tf in the editor

5. Step-by-Step Implementation

Phase 1: Local Environment and Credentials

Step 1 – Create an IAM access key. Sign in to the AWS Management Console and open **IAM → Users → Experiment7**. Under **Security credentials**, click **Create access key**, choose **Command Line Interface (CLI)**, and save the generated Access Key ID and Secret Access Key securely. *(The key values are not reproduced in this report and have been blurred in the screenshots for security.)*

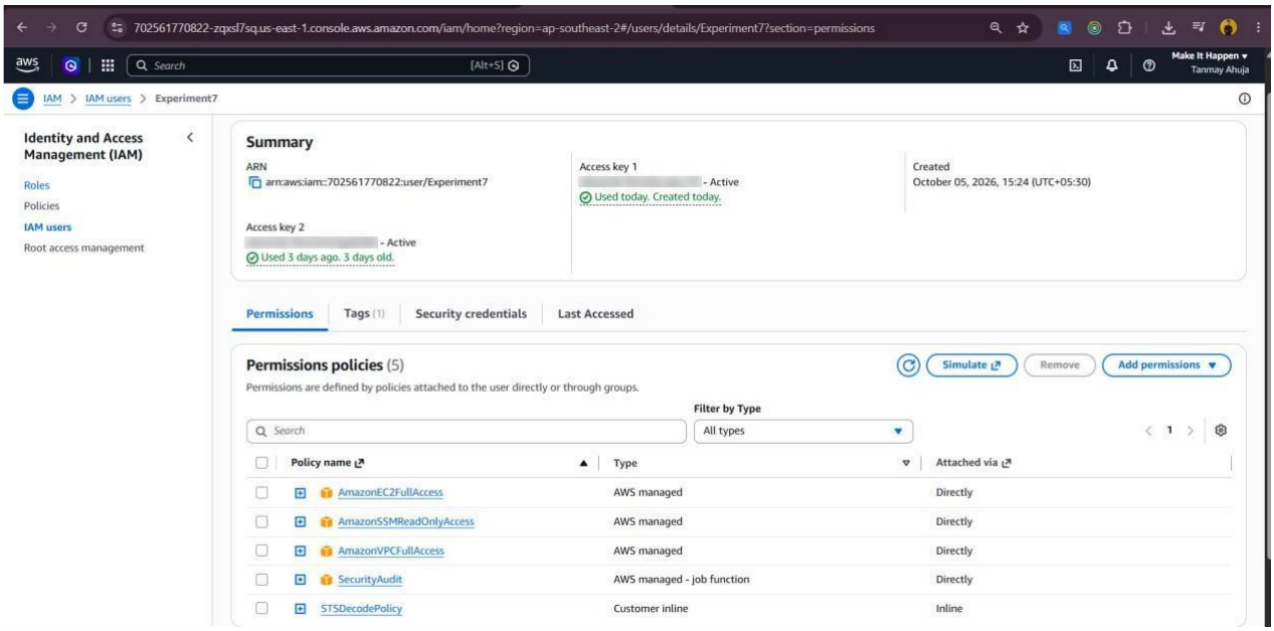


Fig. 5: Access keys for IAM user Experiment7 (key IDs blurred)

Step 2 – Configure and verify the AWS CLI. In Windows PowerShell run **aws configure** and enter the access key, secret key, default region **ap-southeast-2** and output format **json**. Then confirm the identity with:

```
aws configure
aws sts get-caller-identity
```

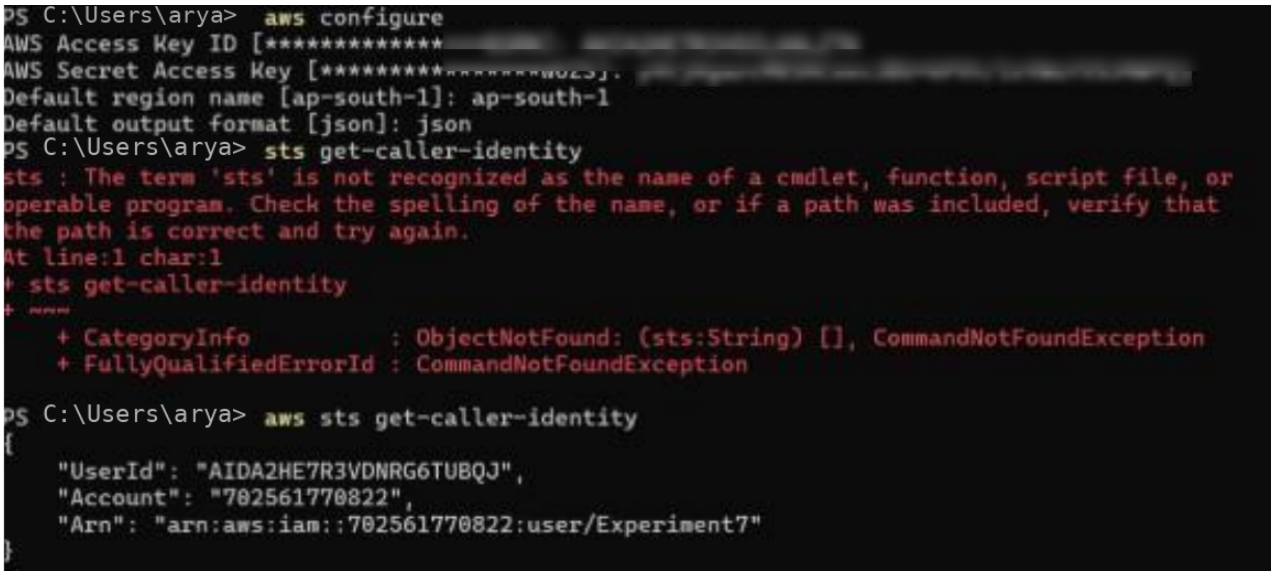


Fig. 6: AWS CLI configuration and STS identity check (keys blurred)

Step 3 – Create an EC2 key pair. In the console go to **EC2 → Key Pairs** (region ap-southeast-2) and click **Create key pair** with name **vmb-key**, type **RSA** and format **.pem**. Download and keep vmb-key.pem safe.

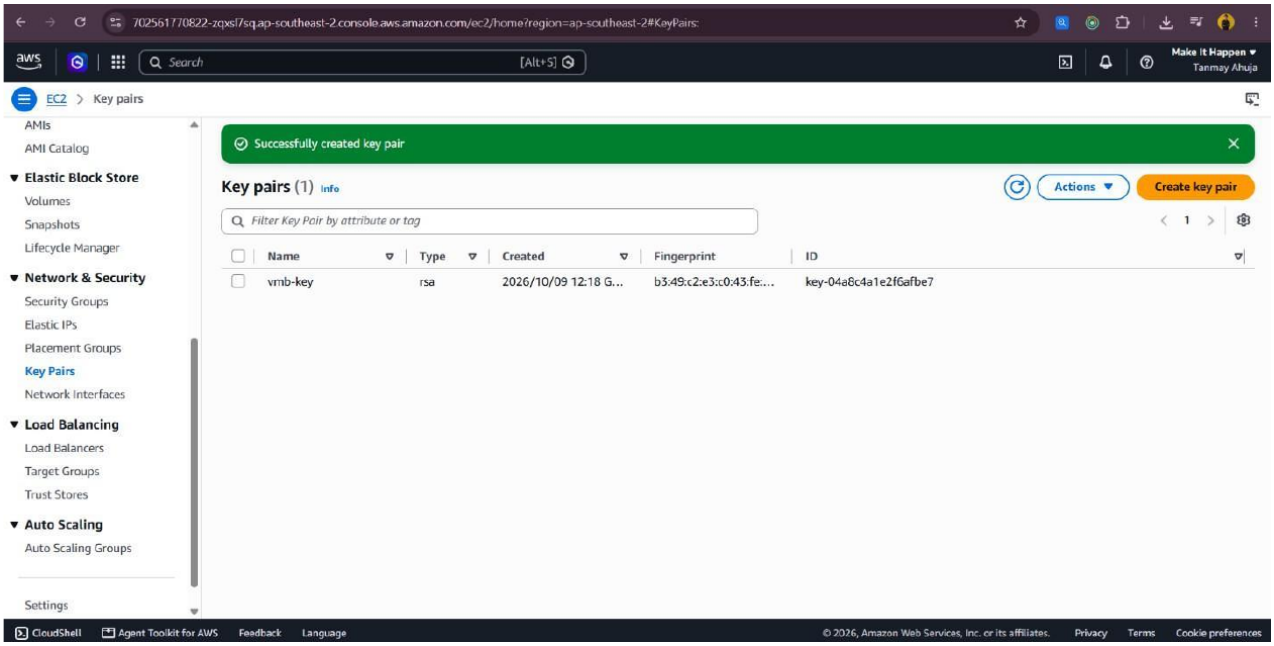


Fig. 7: Key pair vmb-key created in ap-southeast-2

Phase 2: IAM Permission Fix

Step 4 – Attach SSM read-only access. So that Terraform can read the Ubuntu AMI ID from Parameter Store, open **IAM → Users → Experiment7 → Permissions**, choose **Add permissions → Attach policies directly**, select **AmazonSSMReadOnlyAccess** and confirm.

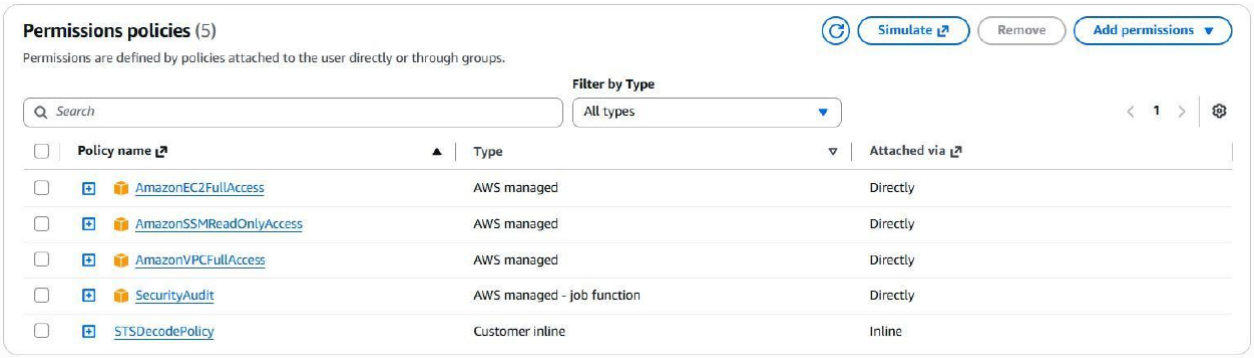


Fig. 8: Policies attached to Experiment7, including AmazonSSMReadOnlyAccess

Phase 3: Terraform Lifecycle

Step 5 – Initialise the working directory. Save main.tf and outputs.tf in C:\Terraform, open PowerShell in that folder and run:

```
cd C:\Terraform
terraform init
```

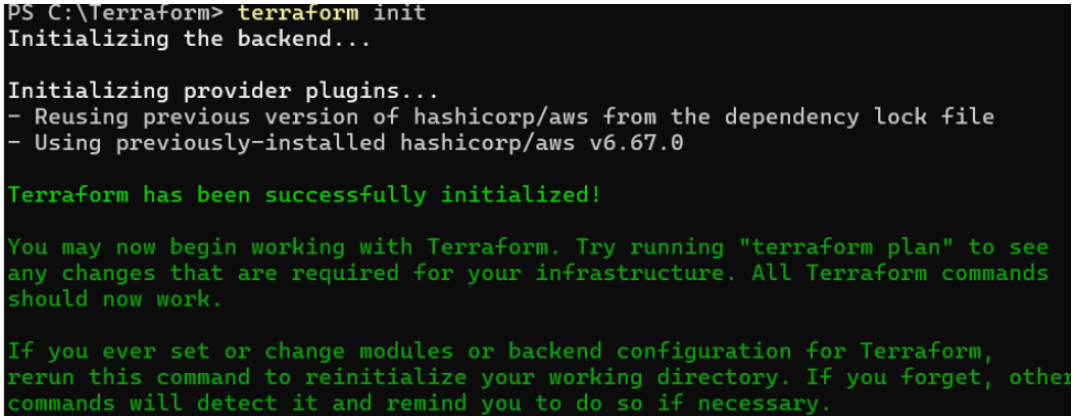


Fig. 9: terraform init completed successfully

Step 6 – Validate and plan. Check the configuration, then preview the changes:

```
terraform validate
terraform plan
```

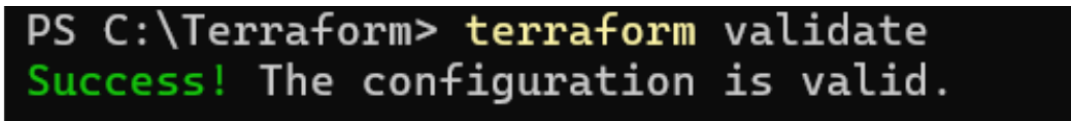


Fig. 10: terraform validate – configuration is valid

```
Windows PowerShell
PS C:\Terraform> terraform plan
data.aws_ssm_parameter.ubuntu_ami: Reading...
data.aws_vpc.default: Reading...
data.aws_ssm_parameter.ubuntu_ami: Read complete after 2s [id=/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id]
data.aws_vpc.default: Read complete after 3s [id=vpc-0e56bf12ccd240a63]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 0s [id=ap-southeast-2]

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.student_vm will be created
  + resource "aws_instance" "student_vm" {
      + ami                                  = (sensitive value)
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "vmb-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
```

Fig. 11: terraform plan output (part 1)

```
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "ap-southeast-2"
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = "subnet-0ecc646eb242b088a"
      + tags                                 = {
          + "Environment" = "Dev"
          + "Name"        = "Terraform-Student-VM"
          + "Project"     = "Experiment7"
        }
      + tags_all                             = {
          + "Environment" = "Dev"
          + "Name"        = "Terraform-Student-VM"
          + "Project"     = "Experiment7"
        }
      + tenancy                              = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + volume_tags                          = {
          + "Environment" = "Dev"
          + "Name"        = "Terraform-Student-VM"
          + "Project"     = "Experiment7"
        }
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)
```

Fig. 12: terraform plan output (part 2)

```
      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id    = (known after apply)
  + public_ip      = (known after apply)

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to
take exactly these actions if you run "terraform apply" now.
```

Fig. 13: Plan summary – 1 to add, 0 to change, 0 to destroy

Step 7 – Apply. Run **terraform apply**, type **yes** at the confirmation prompt and wait for creation to finish.

```
terraform apply
```

```
PS C:\Terraform> terraform apply
data.aws_vpc.default: Reading...
data.aws_ssm_parameter.ubuntu_ami: Reading...
data.aws_ssm_parameter.ubuntu_ami: Read complete after 1s [id=/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id]
data.aws_vpc.default: Read complete after 2s [id=vpc-0e56bf12ccd240a63]
data.aws_subnets.default: Reading...
data.aws_subnets.default: Read complete after 1s [id=ap-southeast-2]

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.student_vm will be created
  + resource "aws_instance" "student_vm" {
      + ami                                  = (sensitive value)
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "vmb-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
```

Fig. 14: terraform apply – proposed changes



Fig. 15: Resource attributes to be created

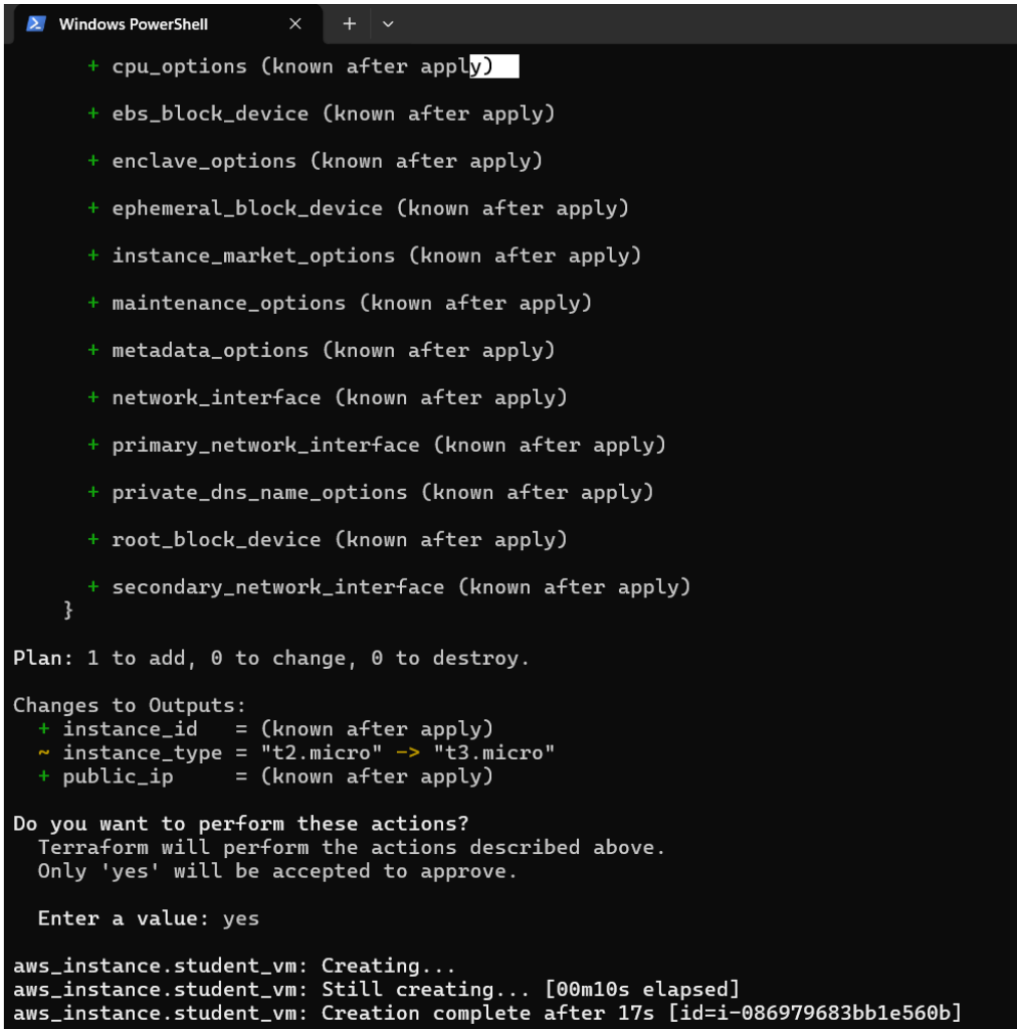


Fig. 16: Approval prompt and instance creation

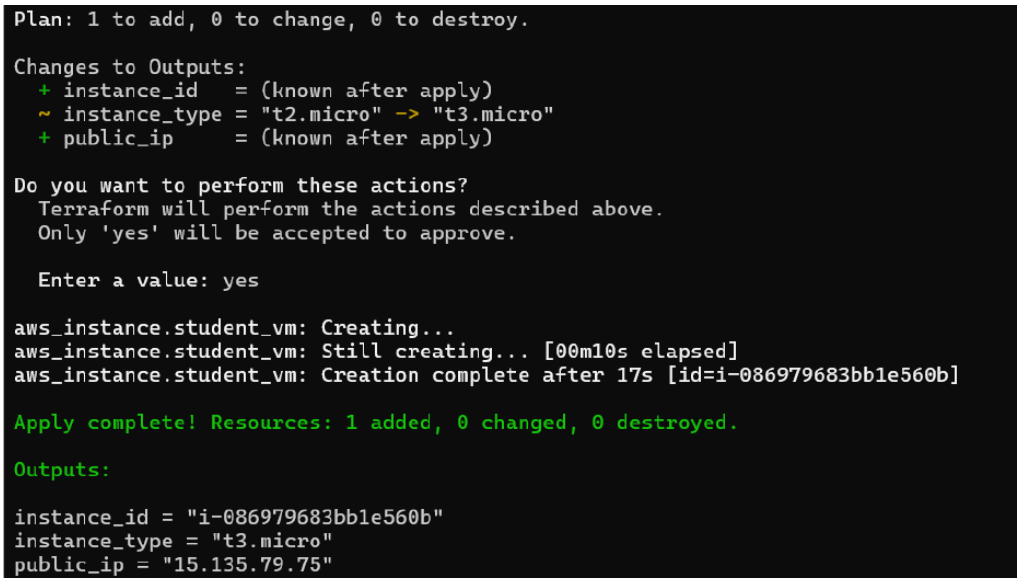


Fig. 17: Apply complete – 1 added, with output values

Phase 4: Verification and Teardown

Step 8 – Check outputs and the EC2 console. Run **terraform output**, then open **EC2 → Instances** in ap-southeast-2 and confirm that **Terraform-Student-VM** (i-086979683bb1e560b) is running with public IP 15.135.79.75.

```
terraform output
```

```
PS C:\Terraform> terraform output
instance_id = "i-086979683bb1e560b"
instance_type = "t3.micro"
public_ip = "15.135.79.75"
```

Fig. 18: terraform output values

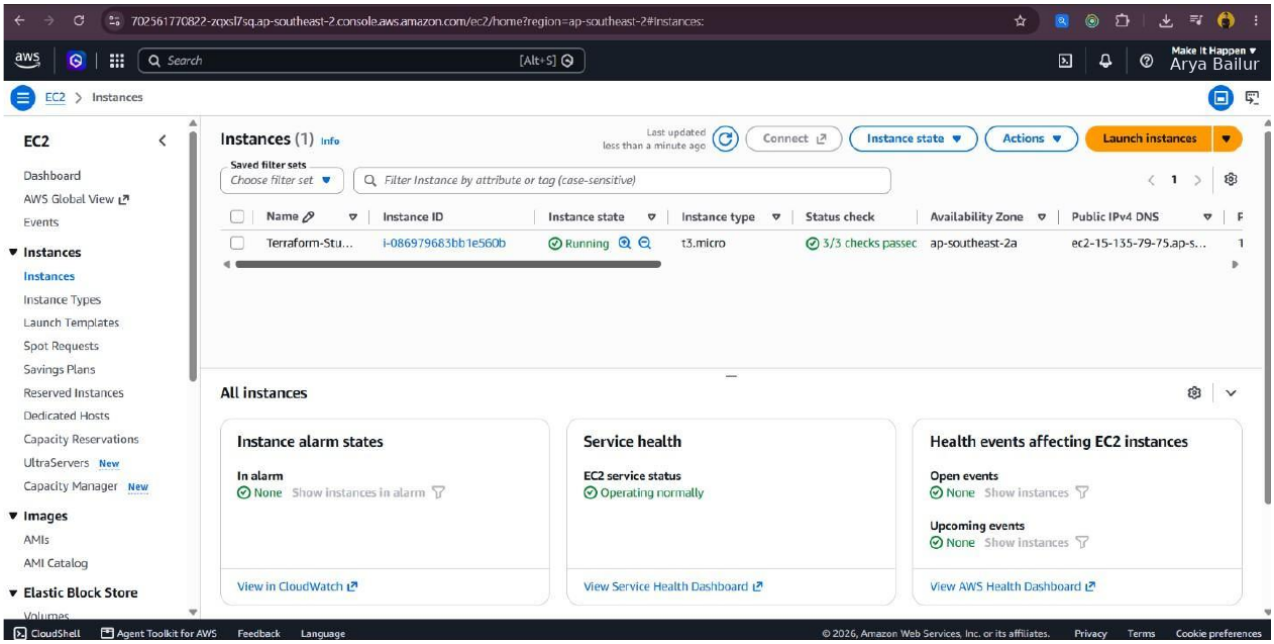


Fig. 19: Terraform-Student-VM running in the EC2 console

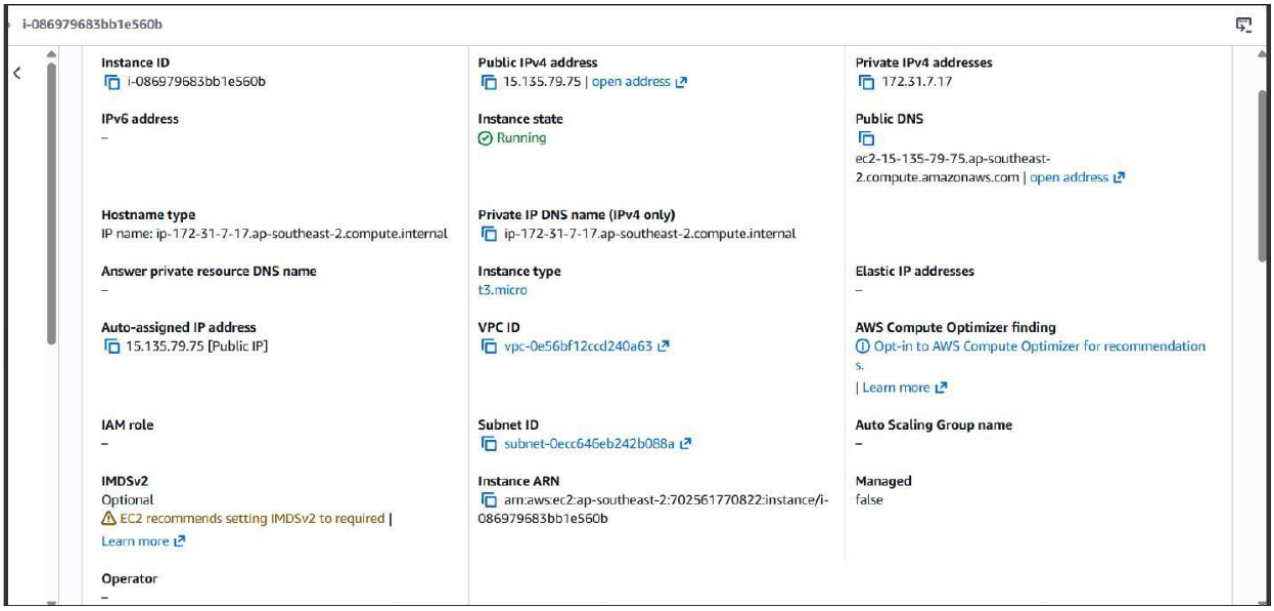


Fig. 20: Instance details – type, IPs, VPC and subnet

Step 9 – Destroy the resources. Run **terraform destroy**, type **yes** when prompted and confirm that everything is removed.

```
terraform destroy
```

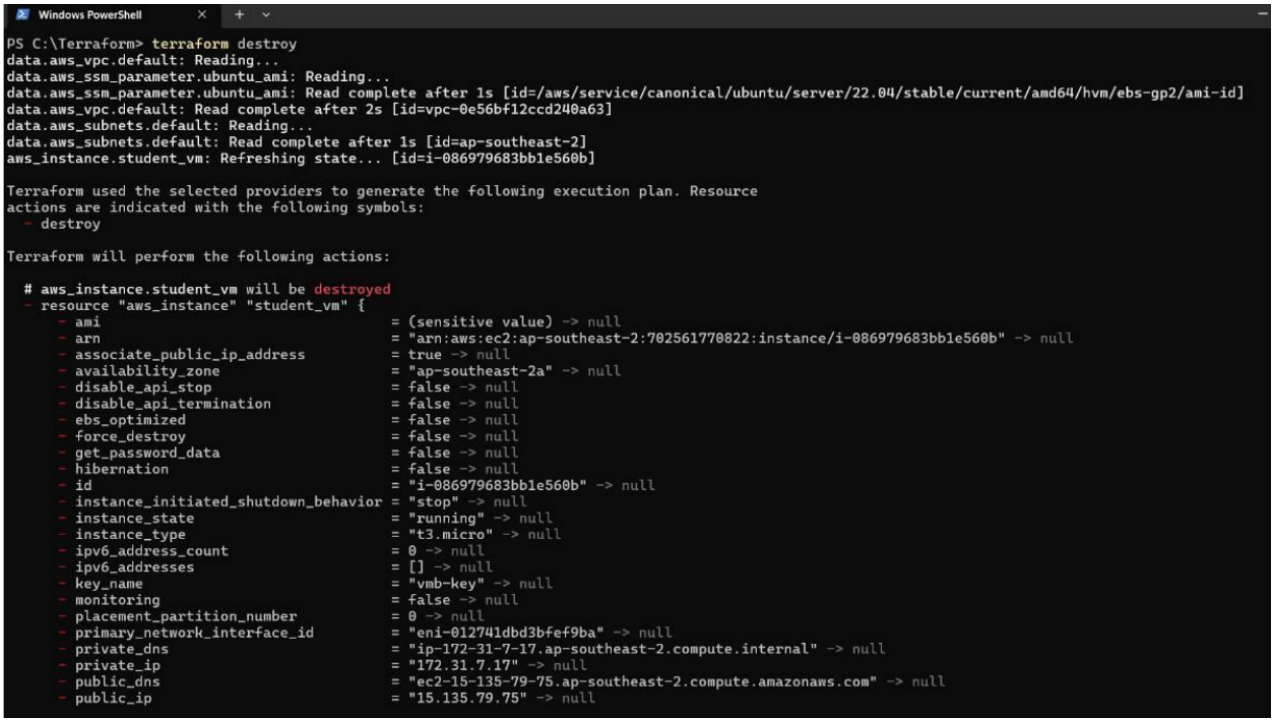


Fig. 21: terraform destroy – refreshing state

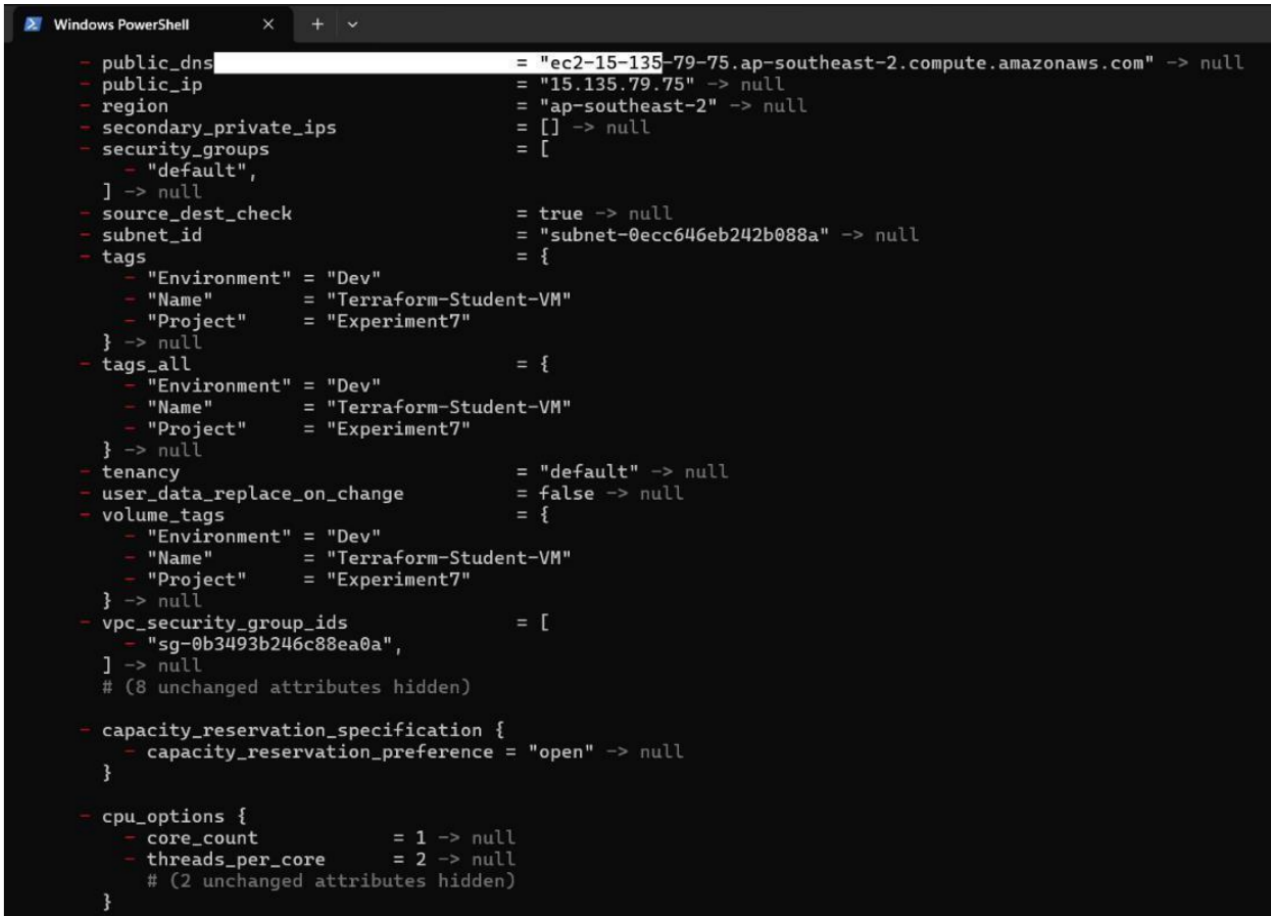


Fig. 22: Resources planned for destruction



Fig. 23: Destroy plan details

```
Windows PowerShell
      - private_dns_name_options {
          - enable_resource_name_dns_a_record    = false -> null
          - enable_resource_name_dns_aaaa_record = false -> null
          - hostname_type                        = "ip-name" -> null
        }

      - root_block_device {
          - delete_on_termination = true -> null
          - device_name           = "/dev/sda1" -> null
          - encrypted             = false -> null
          - iops                  = 100 -> null
          - tags                  = {} -> null
          - tags_all              = {} -> null
          - throughput            = 0 -> null
          - volume_id             = "vol-0f27164e2f72bddb7" -> null
          - volume_size           = 8 -> null
          - volume_type           = "gp2" -> null
            # (1 unchanged attribute hidden)
        }
    }

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  - instance_id   = "i-086979683bb1e560b" -> null
  - instance_type = "t3.micro" -> null
  - public_ip     = "15.135.79.75" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.student_vm: Destroying... [id=i-086979683bb1e560b]
aws_instance.student_vm: Still destroying... [id=i-086979683bb1e560b, 00m10s elapsed]
aws_instance.student_vm: Still destroying... [id=i-086979683bb1e560b, 00m20s elapsed]
aws_instance.student_vm: Destruction complete after 21s

Destroy complete! Resources: 1 destroyed.
PS C:\Terraform>
```

Fig. 24: Destroy complete – 1 destroyed

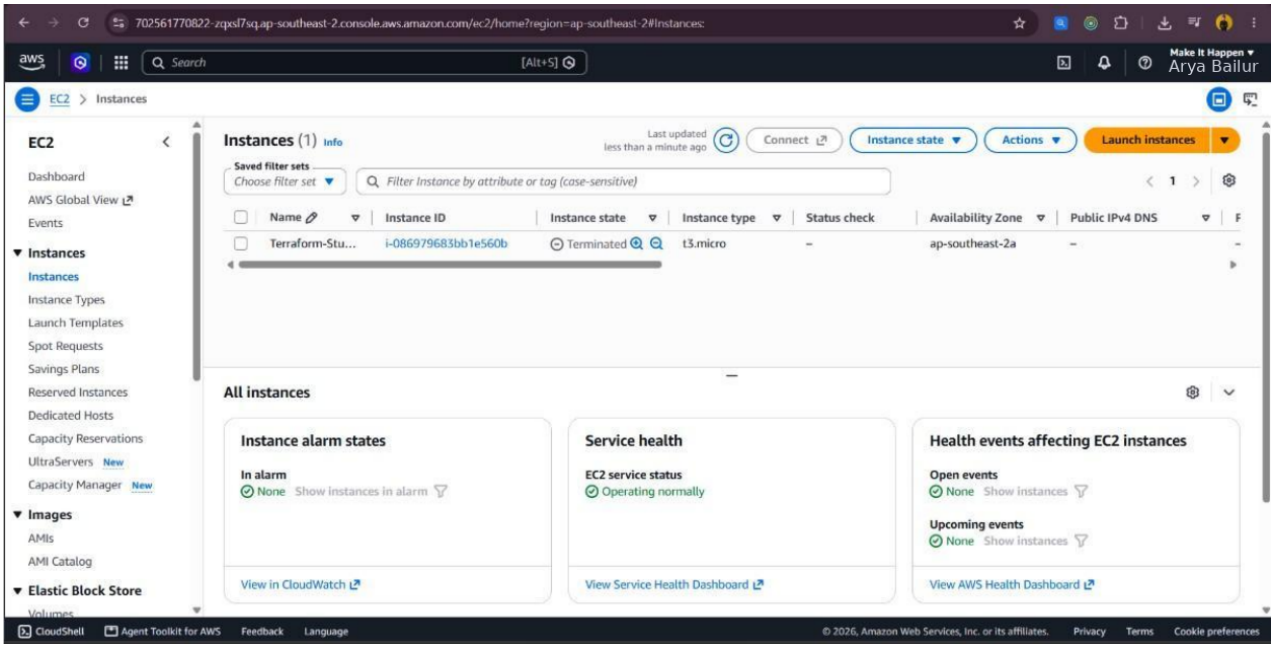


Fig. 25: Instance shown as terminated in the EC2 console

6. Technical Challenges and Solutions

Sr. No.	Error / Issue	Root Cause	Solution
1	403 UnauthorizedOperation on ec2:DescribeImages	The organisation SCP (p-81tb9lnn) blocked AMI search queries.	Replaced the data "aws_ami" block with an SSM Parameter Store lookup (aws_ssm_parameter).
2	AccessDeniedException on ssm:GetParameter	User Experiment7 had no permission to read SSM parameters.	Attached the managed policy AmazonSSMReadOnlyAccess to Experiment7.
3	403 Explicit Deny on ec2:RunInstances	The SCP required tags on both the instance and its root EBS volume.	Added both tags and volume_tags blocks (Environment="Dev", Project="Experiment7").
4	InvalidParameterCombination: not Free Tier eligible	The sandbox only allowed t3.micro, not t2.micro.	Set instance_type = "t3.micro" in main.tf.
5	InvalidAMIID.Malformed	A hard-coded AMI ID did not exist in ap-southeast-2.	Looked up the AMI dynamically for the active region through SSM.

7. Summary of Provisioned Infrastructure

Parameter	Deployed Value	Source / Verification
AWS Account ID	702561770822	aws sts get-caller-identity
IAM User	Experiment7	IAM Console / CLI
AWS Region	ap-southeast-2 (Sydney)	Provider configuration
Instance Name Tag	Terraform-Student-VM	main.tf
Instance ID	i-086979683bb1e560b	terraform output / EC2 Console
Instance Type	t3.micro	EC2 instance details
Public IPv4 Address	15.135.79.75	terraform output / EC2 Console
Private IPv4 Address	172.31.7.17	EC2 instance details
VPC ID	vpc-0e56bf12ccd240a63	VPC Console / data.aws_vpc
Subnet ID	subnet-0ecc646eb242b088a	Subnet Console / data.aws_subnets
Key Pair	vmb-key	EC2 Key Pairs / main.tf
Final Lifecycle State	Terminated	terraform destroy / EC2 Console

8. Conclusion

Learning Outcomes Achieved

This experiment gave hands-on experience of defining, provisioning, managing and tearing down AWS infrastructure with HashiCorp Terraform as an Infrastructure as Code tool. Automating the full resource lifecycle on the cloud satisfied **Lab Outcome 4 (LO4)**.

Industry Applications

- **Automated, repeatable deployments:** declaring infrastructure in version-controlled code removes the human errors that come with manual console work.
- **Multi-cloud management:** Terraform offers one framework to provision resources on AWS, Azure, GCP and Kubernetes using the same declarative HCL style.
- **Policy compliance and governance:** it fits well with enterprise SCPs and security rules, enforcing tagging, network isolation and access control automatically.

Engineering Relevance

Infrastructure as Code is a core part of modern DevOps, Site Reliability Engineering (SRE) and CI/CD pipelines. Managing infrastructure through code lets teams spin up temporary test environments quickly, scale capacity on demand, keep staging and production identical and recover from failures reliably.